

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
FACULTAD DE INGENIERÍA
ARQUI1
LABORATORIO



MANUAL TECNICO

Christopher Miguel Angel Ramos Ascencio
202200057

Requerimientos Minimos del Sistema

Categoría	Requerimiento
Arquitectura	ARM64 (AArch64) o emulación
Sistema Operativo	Linux (Ubuntu recomendado)
Procesador	CPU compatible con AArch64 o soporte para cross-compilation
Memoria RAM	Mínimo 2 GB (recomendado 4 GB)
Espacio en Disco	Al menos 500 MB disponibles
Ensamblador	aarch64-linux-gnu-as
Enlazador	aarch64-linux-gnu-ld
Make	GNU Make
Compilación cruzada	gcc-aarch64-linux-gnu (opcional pero recomendado)
Editor de código	Visual Studio Code u otro editor compatible
Extensiones recomendadas	Soporte para Assembly (ej. ARM Assembly)
Sistema de ejecución	QEMU (si no se cuenta con hardware ARM64)

Descripción General

El presente proyecto implementa un sistema de operaciones de álgebra lineal utilizando Assembly AArch64, trabajando con matrices en formato de punto fijo Q32.32.

El sistema permite realizar operaciones básicas y avanzadas sobre matrices, incluyendo:

- Operaciones aritméticas
- Eliminación Gaussiana
- Gauss-Jordan
- Cálculo de inversa
- Determinante
- División de matrices

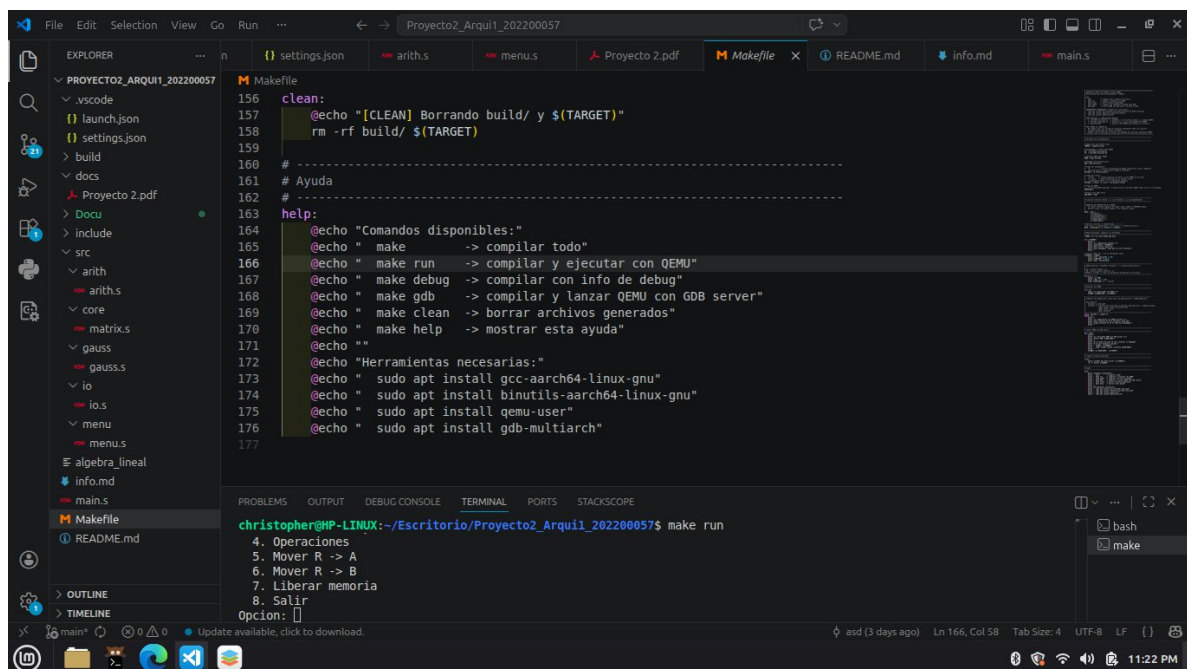
Makefile

El **Makefile** automatiza la construcción del proyecto:

- Ensambla archivos .s a .o
- Crea y Organiza los archivos objeto en la carpeta build/
- Enlaza todos los módulos en el ejecutable final

Herramientas utilizadas:

- aarch64-linux-gnu-as → Ensamblador
- aarch64-linux-gnu-ld → Enlazador



Resumen General Del Sistema

Componente	Función
------------	---------

main.s	Punto de entrada del programa. Inicializa el flujo principal.
menu.s	Maneja la interfaz de usuario y la selección de operaciones.
io.s	Gestiona la entrada y salida (lectura y visualización de datos).
matrix.s	Administra estructuras de matrices (creación, copia, validación, etc.).
arith.s	Implementa operaciones básicas (suma, resta, multiplicación).
gauss.s	Contiene operaciones avanzadas (Gauss, Gauss-Jordan, inversa, etc.).
defines.inc	Define constantes globales y estructuras compartidas.
Makefile	Automatiza el proceso de compilación del proyecto.
build/	Carpeta donde se almacenan los archivos objeto generados (.o).

main.s

Este es nuestro archivo main en la carpeta raíz de nuestro proyecto de archi, De aca parte el programa desde el punto de inicio definido comunmente como `_start` Este programa da pie la inicializacion de las matrices y mostrar el menu principal

Funcionalidades:

- Punto de entrada del programa (`_start`)
- Inicializa la ejecución
- Invoca el menú principal

```

40  start:
41
42  // -----
43  // PASO 3: Inicializar los descriptores de matrices A, B y R.
44  // Esto pone STATUS_FREE en los tres descriptores.
45  // Aunque .bss ya viene en cero, llamar matrix_init es una buena practica
46  // porque hace el codigo mas legible y explicito.
47  // -----
48  bl matrix_init          // inicializar A, B, R -> STATUS_FREE
49
50  // -----
51  // PASO 4: Entrar al bucle principal del menu.
52  // menu main no retorna hasta que el usuario elige "Salir" (opcion 8).
53  // Antes de retornar, menu main llama matrix_free_all para liberar memoria.
54  // -----
55  bl menu_main            // bucle del menu (no retorna hasta salir)
56
57  // -----
58  // PASO 5: Salir del proceso con codigo de salida 0 (exito).
59  // Llamamos a la syscall exit directamente.
60  // x0 = codigo de salida (0 = exito)
61  // x8 = numero de syscall (SYS_EXIT = 93 en ARM64 Linux)
62  // svc #0 = llamada al kernel
63  // NOTA: Esta instruccion nunca retorna, El kernel termina el proceso.
64  svc #0
65
66  .endif

```

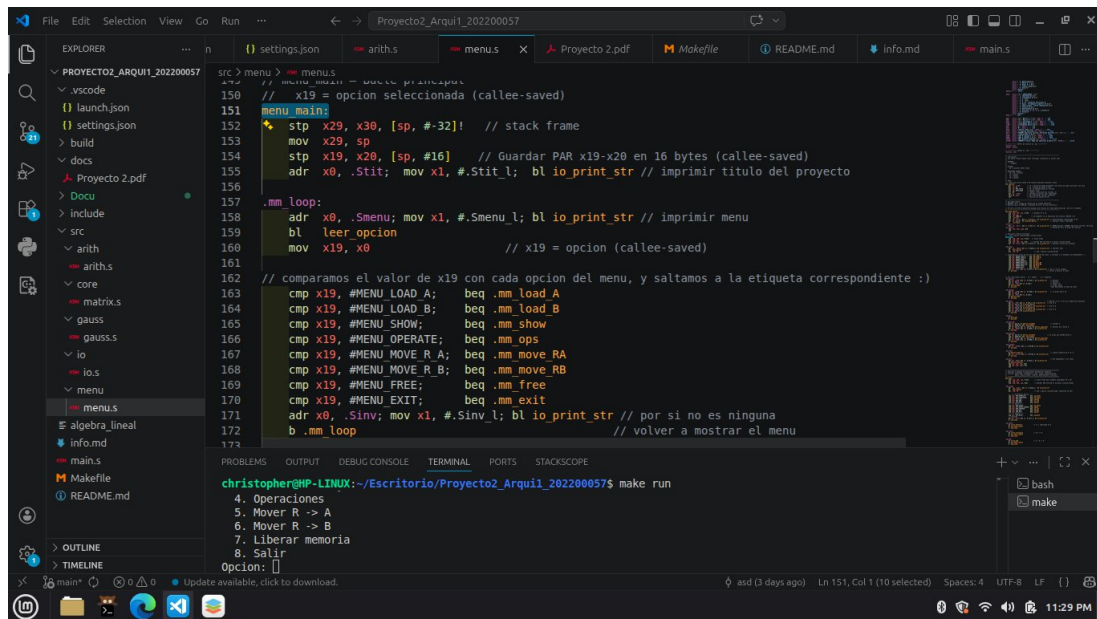
christopher@P-LINUX:~/Escritorio/Proyecto2_Arqui_202200057\$ make run

4. Operaciones
5. Mover R -> A
6. Mover R -> B
7. Liberar memoria
8. Salir
Option: 1

menu.s

Esta clase se encarga principalmente de la gestion del menu principal donde se cargan muestran las matrices y tambien del submenu el cual es para las operaciones
Funcionalidades:

- Gestiona la interacción con el usuario
- Muestra las opciones disponibles
- Redirige a la operación seleccionada



```
src > menu > menu.s
// menu_main = func principal
150 // x19 = opcion seleccionada (callee-saved)
151 menu_main:
152     stp x29, x30, [sp, #-32]! // stack frame
153     mov x29, sp
154     stp x19, x20, [sp, #16] // Guardar PAR x19-x20 en 16 bytes (callee-saved)
155     adr x0, .Stit; mov x1, #.Stit_l; bl io_print_str // imprimir titulo del proyecto
156
157 .mm loop:
158     adr x0, $.menu; mov x1, $.Smenu_l; bl io_print_str // imprimir menu
159     bl leer_opcion
160     mov x19, x0 // x19 = opcion (callee-saved)
161
162 // comparamos el valor de x19 con cada opcion del menu, y saltamos a la etiqueta correspondiente :)
163 cmp x19, #MENU_LOAD_A; beq .mm load_A
164 cmp x19, #MENU_LOAD_B; beq .mm load_B
165 cmp x19, #MENU_SHOW; beq .mm show
166 cmp x19, #MENU_OPERATE; beq .mm ops
167 cmp x19, #MENU_MOVE_R_A; beq .mm move_RA
168 cmp x19, #MENU_MOVE_R_B; beq .mm move_RB
169 cmp x19, #MENU_FREE; beq .mm free
170 cmp x19, #MENU_EXIT; beq .mm exit
171 adr x0, $.Sinv; mov x1, $.Sinv_l; bl io_print_str // por si no es ninguna
172 b .mm loop // volver a mostrar el menu
173
```

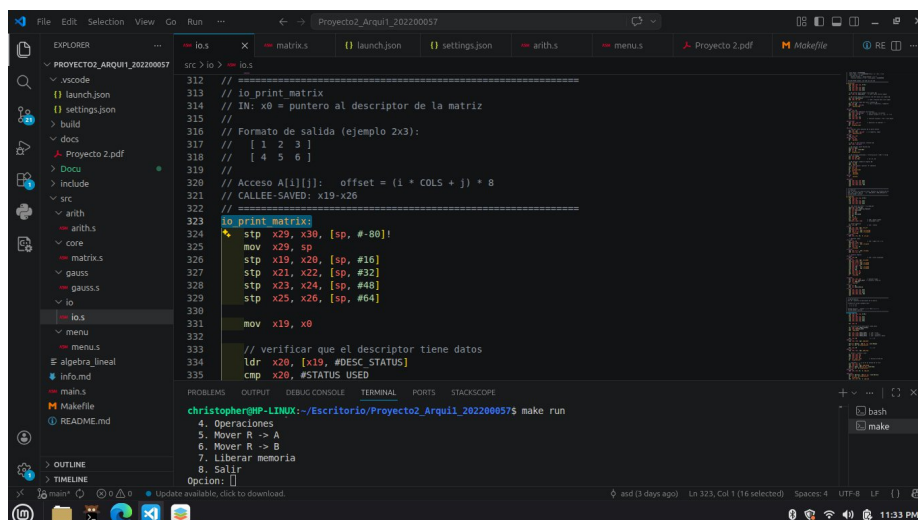
christopher@HP-LINUX:~/Escritorio/Proyecto2_Arqui1_202200057\$ make run

```
4. Operaciones
5. Mover R -> A
6. Mover R -> B
7. Liberar memoria
8. Salir
Opcion: [
```

io.s

Esta clase se encarga de todo lo que tiene que ver con la impresion de los datos en la consola. Cuando necesitamos mostrar algo en la terminal quien se encargara de esto son las funciones de io (input output)

- Manejo de entrada y salida
- Lectura de datos desde teclado
- Impresión de matrices y mensajes
- Uso de syscalls en Linux AArch64



```
src > io > io.s
312 // =====
313 // io_print_matrix
314 // IN: x0 = puntero al descriptor de la matriz
315 //
316 // Formato de salida (ejemplo 2x3):
317 // [ 1 2 3 ]
318 // [ 4 5 6 ]
319 //
320 // Acceso A[i][j]: offset = (i * COLS + j) * 8
321 // CALLEE-USED: x19-x26
322 // =====
323 io_print_matrix:
324     stp x29, x30, [sp, #-80]!
325     mov x29, sp
326     stp x19, x20, [sp, #16]
327     stp x21, x22, [sp, #32]
328     stp x23, x24, [sp, #48]
329     stp x25, x26, [sp, #64]
330
331     mov x19, x0
332
333 // verificar que el descriptor tiene datos
334 ldr x20, [x19, #DESC_STATUS]
335 cmp x20, #STATUS_USED
```

christopher@HP-LINUX:~/Escritorio/Proyecto2_Arqui1_202200057\$ make run

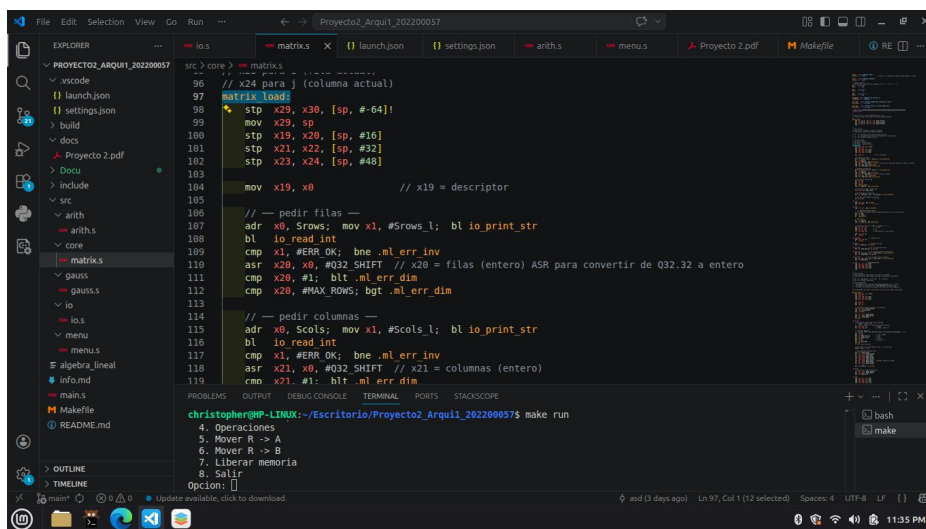
```
4. Operaciones
5. Mover R -> A
6. Mover R -> B
7. Liberar memoria
8. Salir
Opcion: [
```

matrix.s

Clase que basicamente se encarga desde la inicializacion hasta la carga y liberacion de la memoria cuando se carga una matriz A B o resultante

Funcionalidades:

- Administración de estructuras de matrices
- Funciones de la clase principales:
 - o matrix_resize: Reserva o redimensiona memoria
 - o matrix_free: Libera memoria
 - o matrix_validate: Verifica integridad de la matriz
 - o matrix_copy_desc: Copia descriptores de matrices
- Controla dimensiones, elementos y memoria asociada



```
src > core > matrix.s
96 // x24 para j (columna actual)
97 matrix_load:
98     stp x29, x30, [sp, #-64]!
99     mov x29, sp
100     stp x19, x20, [sp, #16]
101     stp x21, x22, [sp, #32]
102     stp x23, x24, [sp, #48]
103
104     mov x19, x0 // x19 = descriptor
105
106 // -- pedir filas --
107 adr x0, $rows; mov x1, #rows_l; bl io_print_str
108 bl io_read_int
109 cmp x1, #ERR_OK; bne .ml_err_inv
110 asr x20, x0, #Q32_SHIFT // x20 = filas (entero) ASR para convertir de Q32.32 a entero
111 cmp x20, #1; blt .ml_err_dim
112 cmp x20, #MAX_ROWS; bgt .ml_err_dim
113
114 // -- pedir columnas --
115 adr x0, $cols; mov x1, #cols_l; bl io_print_str
116 bl io_read_int
117 cmp x1, #ERR_OK; bne .ml_err_inv
118 asr x21, x0, #Q32_SHIFT // x21 = columnas (entero)
119 cmn x21, #1; blt .ml_err_dim
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS STACKSCOPE

christopher@WP-LINUX:~/Escritorio/Proyecto2_Arqui1_202200057\$ make run

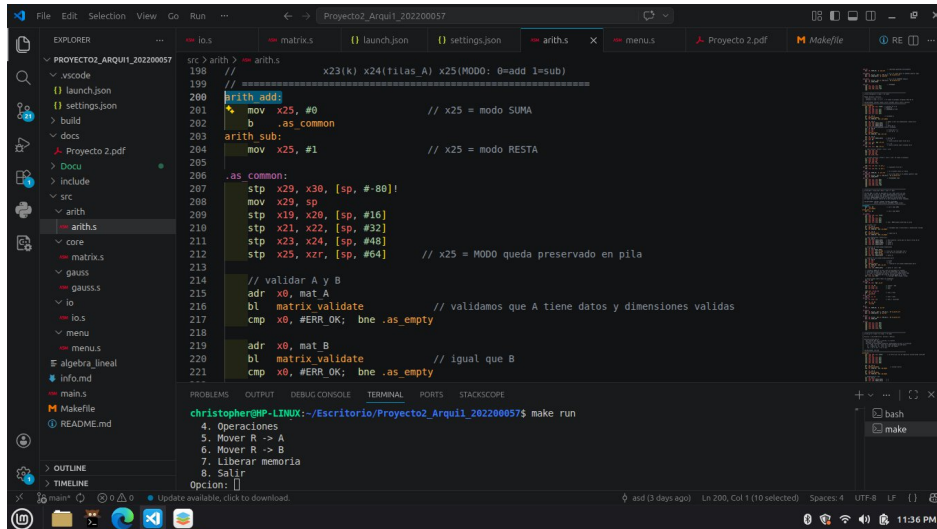
```
4. Operaciones
5. Mover A -> A
6. Mover B -> B
7. Liberar memoria
8. Salir
Opciones []
```

arith.s

Clase que se encarga de las operaciones “fáciles” las cuales son la resta suma y multiplicación

Funcionalidades:

- Implementa operaciones básicas:
 - Suma ($A + B$)
 - Resta ($A - B$)
 - Multiplicación ($A * B$)
- Opera directamente sobre los datos de las matrices



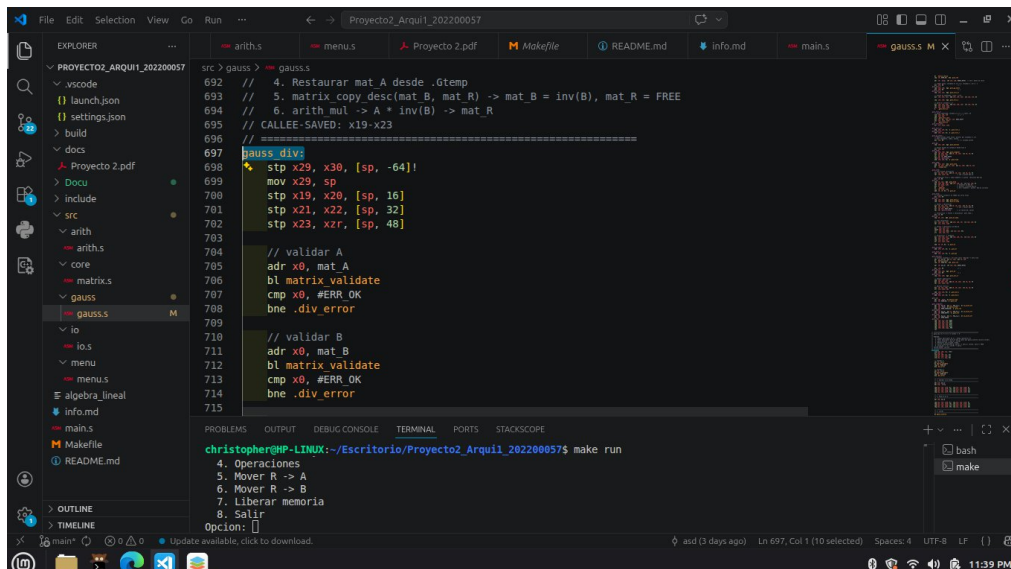
```
src> arith> arith.s
198 // x23(k) x24(tilas_A) x25(MODO: 0=add 1=sub)
199 // =====
200 arith_add:
201     mov x25, #0 // x25 = modo SUMA
202     b .as_common
203 arith_sub:
204     mov x25, #1 // x25 = modo RESTA
205
206 .as_common:
207     stp x29, x30, [sp, #-80]!
208     mov x29, sp
209     stp x19, x20, [sp, #16]
210     stp x21, x22, [sp, #32]
211     stp x23, x24, [sp, #48]
212     stp x25, xzr, [sp, #64] // x25 = MODO queda preservado en pila
213
214 // validar A y B
215 adr x0, mat_A
216 bl matrix_validate // validamos que A tiene datos y dimensiones validas
217 cmp x0, #ERR_OK; bne .as_empty
218
219 adr x0, mat_B
220 bl matrix_validate // igual que B
221 cmp x0, #ERR_OK; bne .as_empty
222
christopher@HP-LINUX:~/Escritorio/Proyecto2_Arqui1_202200057$ make run
4. Operaciones
5. Mover R -> A
6. Mover R -> B
7. Liberar memoria
8. Salir
Opcion: [ ]
```

gauss.s

La clase mas larga y compleja, encargada unica y exclusivamente de las operaciones gaussianas.

Funcionalidades:

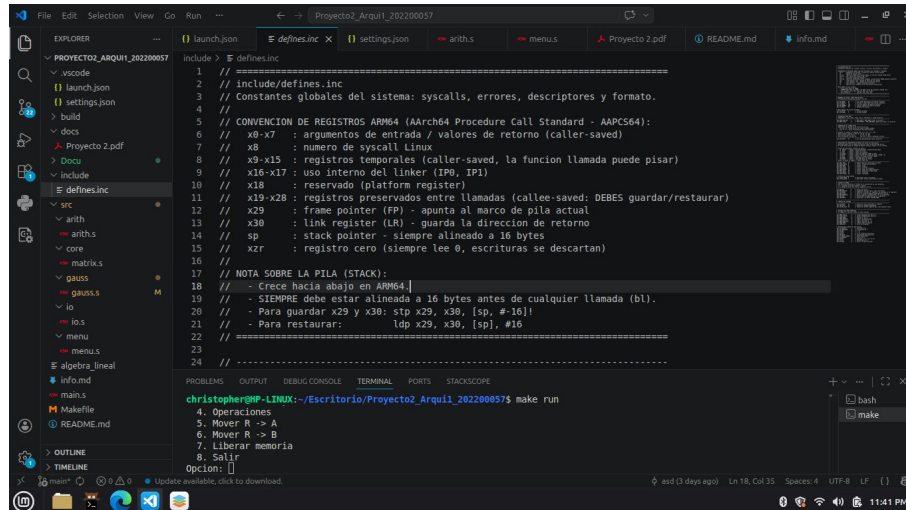
- gauss_eliminate: Forma triangular superior
- gauss_jordan: Forma reducida por filas (RREF)
- gauss_inverse: Cálculo de la inversa
- gauss_determinant: Cálculo del determinante
- gauss_div: División de matrices ($A * \text{inv}(B)$)



```
src> gauss> gauss.s
692 // 4. Restaurar mat A desde .Gtemp
693 // 5. matrix copy desc(mat_B, mat_R) -> mat_B = inv(B), mat_R = FREE
694 // 6. arith_mul -> A * inv(B) -> mat_R
695 // CALLEE- SAVED: x19-x23
696 // =====
697 gauss_div:
698     stp x29, x30, [sp, #-64]!
699     mov x29, sp
700     stp x19, x20, [sp, 16]
701     stp x21, x22, [sp, 32]
702     stp x23, xzr, [sp, 48]
703
704 // validar A
705 adr x0, mat_A
706 bl matrix_validate
707 cmp x0, #ERR_OK
708 bne .div_error
709
710 // validar B
711 adr x0, mat_B
712 bl matrix_validate
713 cmp x0, #ERR_OK
714 bne .div_error
715
christopher@HP-LINUX:~/Escritorio/Proyecto2_Arqui1_202200057$ make run
4. Operaciones
5. Mover R -> A
6. Mover R -> B
7. Liberar memoria
8. Salir
Opcion: [ ]
```


defines.inc

- Archivo de definiciones globales
- Contiene:
 - Constantes del sistema (ej. Q32_SHIFT)
 - Offsets de estructuras (DESC_ROWS, DESC_DATA, etc.)
 - Códigos de error (ERR_OK, ERR_EMPTY, etc.)
- Garantiza consistencia entre módulos



```
1 // =====
2 // include/defines.inc
3 // Constantes globales del sistema: syscalls, errores, descriptors y formato.
4 //
5 // CONVENCIÓN DE REGISTROS ARMv64 (AArch64 Procedure Call Standard - AAPCS64):
6 // x0-x7 : argumentos de entrada / valores de retorno (caller-saved)
7 // x8 : numero de syscall linux
8 // x9-x15 : registros temporales (caller-saved, la función llamada puede pisar)
9 // x16-x17 : uso interno del linker (IP0, IP1)
10 // x18 : reservado (platform register)
11 // x19-x28 : registros preservados entre llamadas (callee-saved: DEBES guardar/restaurar)
12 // x29 : frame pointer (FP) - apunta al marco de pila actual
13 // x30 : link register (LR) - guarda la dirección de retorno
14 // sp : stack pointer - siempre alineado a 16 bytes
15 // xzr : registro cero (siempre lee 0, escrituras se descartan)
16 //
17 // NOTA SOBRE LA PILA (STACK):
18 // - Crece hacia abajo en ARMv64.
19 // - SIEMPRE debe estar alineada a 16 bytes antes de cualquier llamada (bl).
20 // - Para guardar x29 y x30: stp x29, x30, [sp, #-16]!
21 // - Para restaurar: ldp x29, x30, [sp], #16
22 // =====
23
24 //
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS STACKSCOPE

christopher@HP-LINUX:~/Escritorio/Proyecto2_Arquili_202200057\$ make run

4. Operaciones

5. Mover R -> A

6. Mover R -> B

7. Liberar memoria

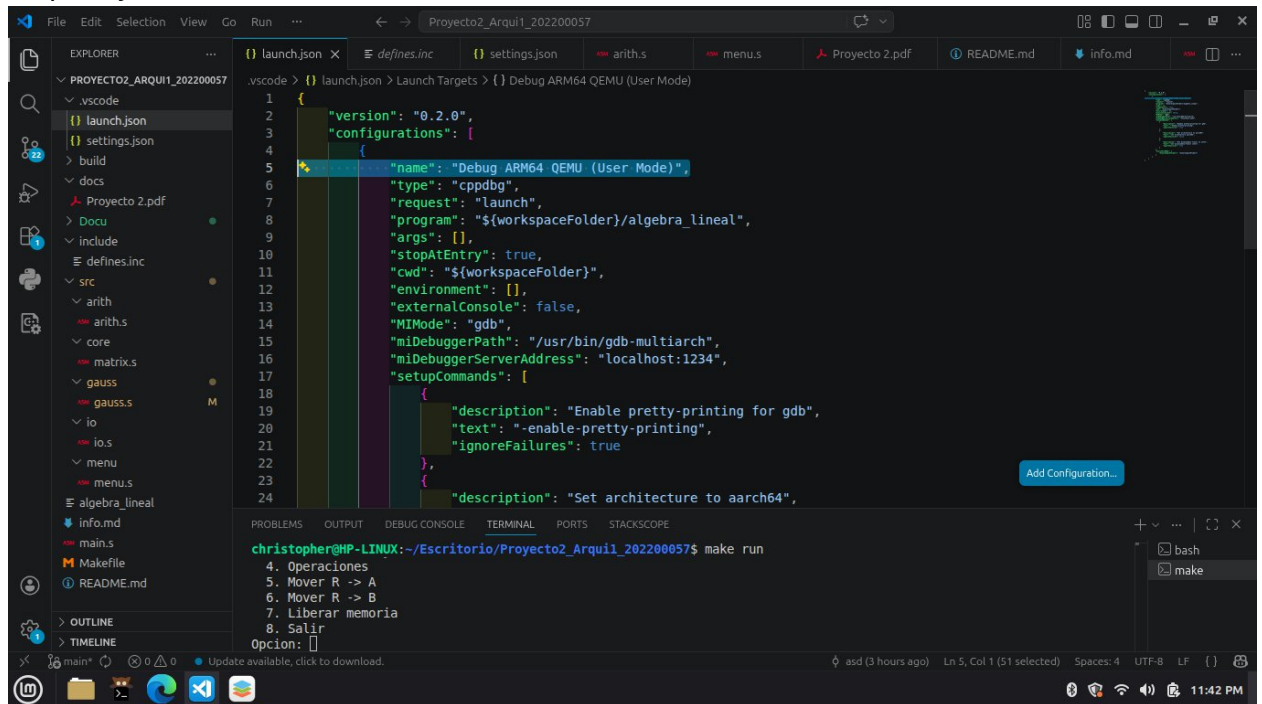
8. Salir

Opcion: []

DEBUGGER

launch.json

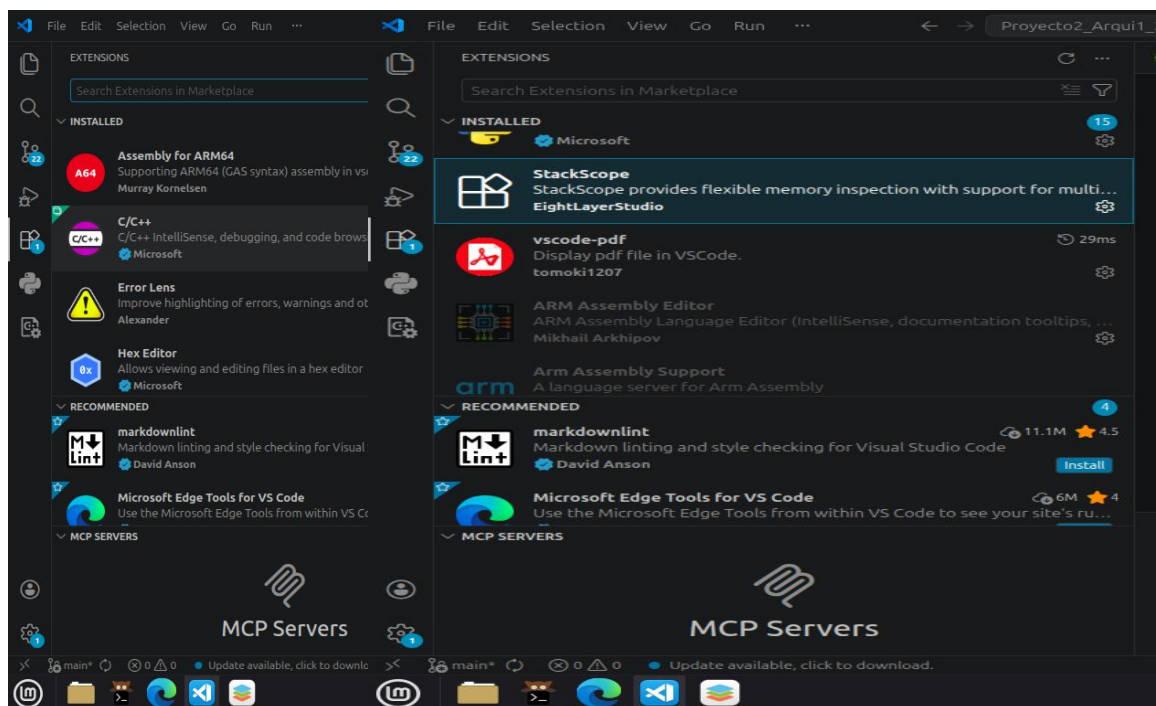
Esta clase se encarga directamente de la configuracion de el debugger con qemu o con raspberry



Requisitos para Debuggear:

El principal requisito para debuggear es tener dos librerias

C/c++ y alguna para ver registros de memoria como StackScope



Pasos para debuggear

- 1) en la terminal escribe “make gdb”
- 2) Selecciona los BreakPoints
- 3) a la izquierda de tu programa selecciona “Run and Debug”
debes ver algo así:

